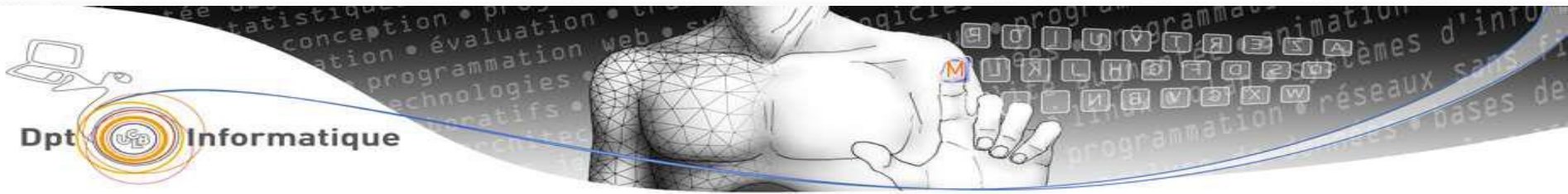
UPSSA
UNIVERSITÉ PRIVÉE DE LA SANTÉ ET DES SCIENCES D'AGADIR
الجامعة الخاصة للصحة و العلوم لأكادير

PROGRAMMATION ORIENTÉE OBJET EN JAVA

Said CHARFI
Maitre de conférences habilité
charfisaid@gmail.com/s.charfi@uiz.ac.ma

Filière : Génie Informatique



Plan

- Bases et les concepts de JAVA
- Concepts de l'orienté objet; classes, méthodes…..
- Héritage et Polymorphisme
- Classes Abstract, Interfaces, classes génériques - Enumération haines de caractères
- Gestions des erreurs avec des exceptions
- Interfaces Graphiques
- Applets, entrées/sorties….

Présentation de JAVA

Bases de Java

Historique

- 1991, chez SUN est créé le langage OAK (pour interface entre appareils ménagers et ordinateurs)
- 1994, Oak se tourne vers Internet et devient Java
- 1995, Java est intégré à Netscape
- 1998, Version 2 de Java
- 2000, Version 1.3 de Java
- 2002, Version 1.4 de Java
- 2004, Java 5
- 2011, Java JDK 7
- 2014, Java 8 (LTS)
- 2018, Java 11 (LTS)
- 2021, Java 17 (LTS)
- 2023, Java 21 (LTS)
- 2025, Java 25 (LTS, dernière version à support long terme)
- 2026, Java 27 (dernière version, septembre 2026)

Présentation

- Java est un langage récent qui s'est répandu de façon étonnamment rapide

- Car : Java est portable, non dépendant de l'environnement d'implémentation (de la machine)

- Car : la syntaxe de Java est très proche du langage C++ (et donc de C)…

Présentation

- Car : Java est robuste
  - Gestion de la mémoire par un système de ramasse - miettes (Garbage Collector)
  - Typage fort
- Car : Il existe un grand nombre de classes sûres et commentées
- Car Java est gratuit …

Présentation

- Java est "vraiment" Orienté Objet :
  tout est objet
- Les programmes java sont multi plate-forme
- Langage haute sécurité
- Java est utilisé pour créer des programmes et des Applets
- Java fournit un code (ByteCode) interprété par une machine virtuelle
  - (+) Garantit la portabilité
  - (-) Ralentit l'exécution

Télécharger Java

- Sur le site (www.oracle.com), Oracle propose les différents Kits de Développement Java (JDK)
- https://www.oracle.com/technetwork/java/javase/downloads/index.html

- Sur Internet, grand nombre de sources et de documentation
  [Ex : "Thinking in Java" de B. Eckel]
  http://sd.blackball.lv/library/Thinking_in_Java_4th_edition.pdf



- Programmer en Java (2ème édition)
  - Auteur : Claude Delannoy
  - Éditeur : Eyrolles
  - Edition : 2002 - 661 pages - ISBN : 2212111193



- Java en action
  - Auteur : Ian F. Darwin
  - Éditeur : O'Reilly
  - Edition : 2002 - 836 pages - ISBN : 2841772039



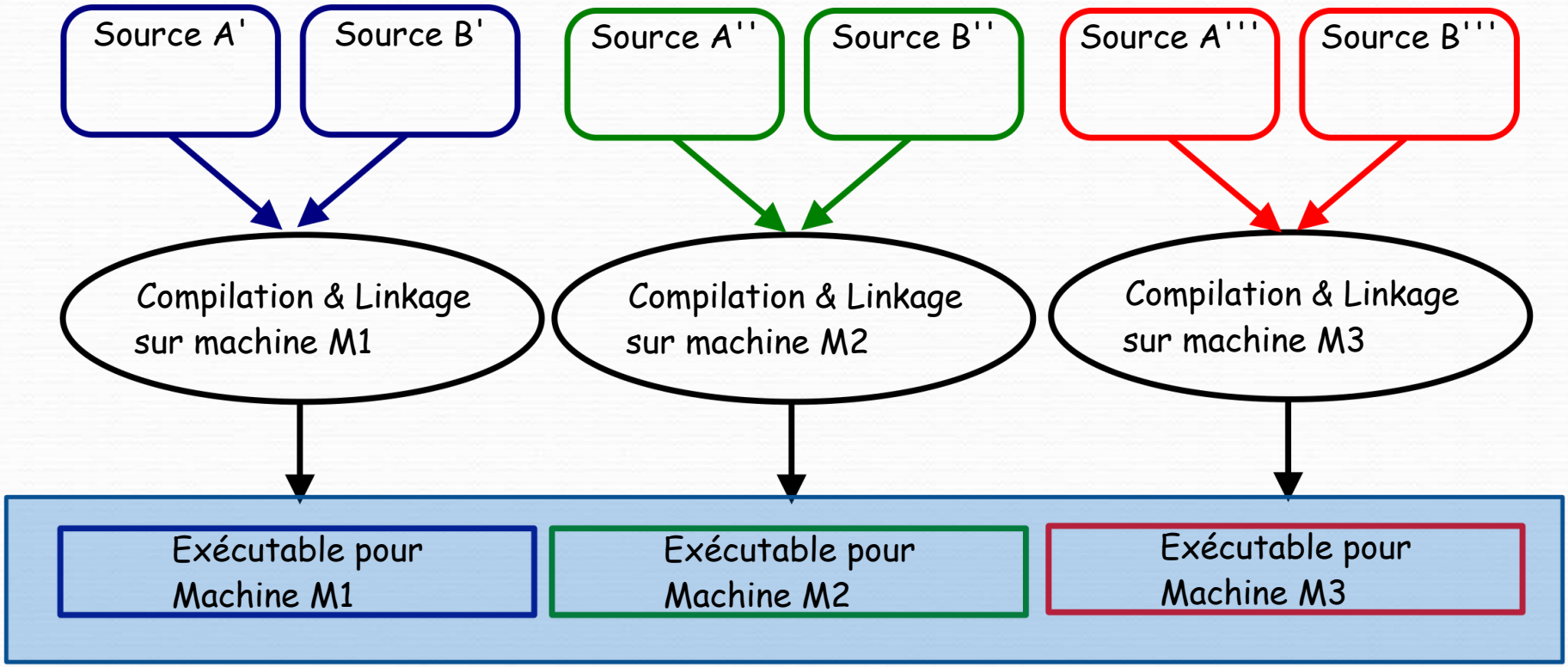
- Apprendre Java et C++ en parallèle
  - Auteur : Jean-Bernard Boichat
  - Éditeur : Eyrolles
  - Edition : 2003 - 742 pages - ISBN : 2212113277

Les concepts de JAVA

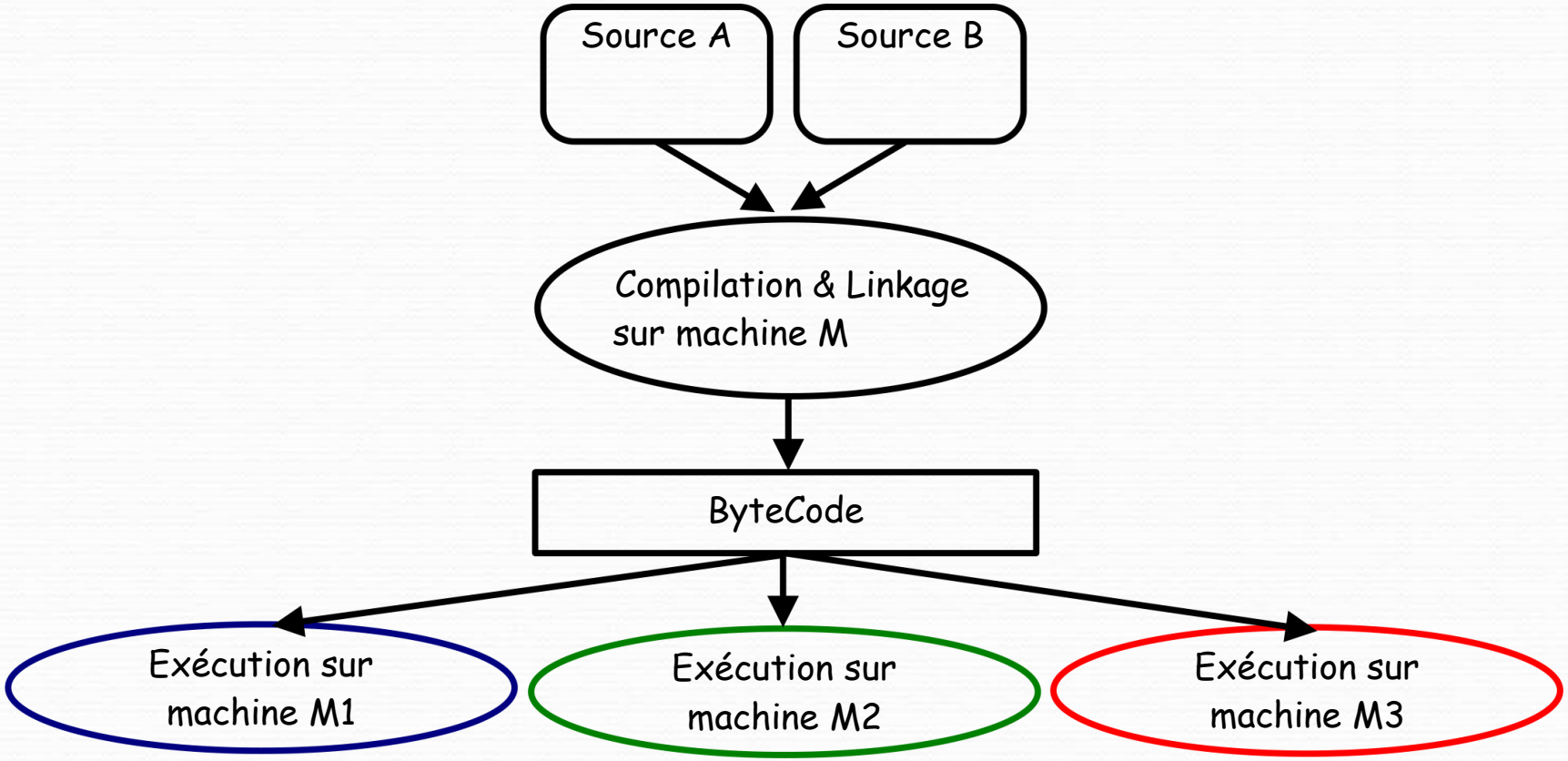
La Compilation & JAVA

- Le principal atout de JAVA concerne sa portabilité.
- Celle-ci est possible par la notion de ByteCode.
- JAVA ne produit pas un code natif, directement exécutable.
- JAVA produit un code intermédiaire, le ByteCode, interprétable par une machine virtuelle.

La Compilation "classique"



La Compilation Java



Interprétation par machine virtuelle (Java Virtual Machine)
spécifique au système d'exploitation

Code natif vs ByteCode

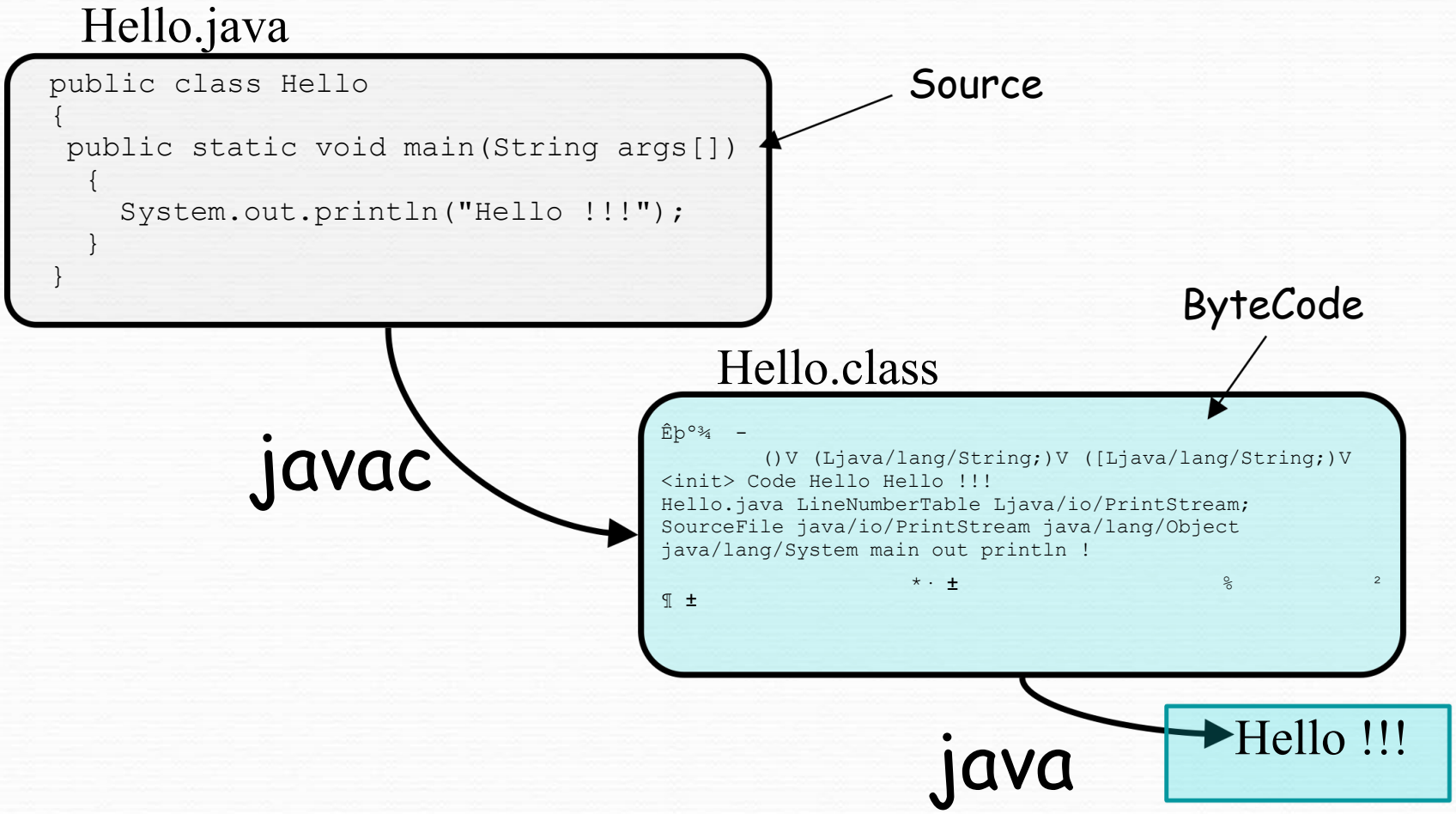
- (+) rapidité d'exécution
- (-) source pas toujours portable
- (-) recompiler pour changer de système

- (+) source portable
- (+) Bytecode portable
- (-) moins rapide car interprété

Source portable

- Les types primitifs sont totalement spécifiés.
- Ils ne dépendent pas de l'architecture de la machine :
  - Les entiers, réels, etc… sont toujours définis sur le même nombre de bits.
- Génération du ByteCode par le compilateur javac

Exemple de compilation



Ramasse Miettes

- En JAVA, tout est objet. Il faut donc créer explicitement chaque objet manipulé.
- Cependant, il est inutile de les détruire. Le Garbage Collector (ramasse miettes) s'en charge.
- Dès qu'un objet n'est plus utilisé (sortie de boucle, ...), il est détruit.

Eléments de base

Tout est Objet (ou presque...)

- Tous les éléments en JAVA dérivent de la classe Object.
- Tous sauf les primitives (8 en tout) représentant des types simples.
- A chaque type de primitive correspond une classe d'objet.

Les primitives : des types simples

Nom	*Taille*	*Minimum*	*Maximum*	*Classe*
boolean	1 bit	false	true	Boolean
char	16 bits	Unicode 0	Unicode $2^{16}-1$	Character
byte	8 bits	-128	127	Byte
short	16 bits	-2^{15}	$2^{15}-1$	Short
int	32 bits	-2^{31}	$2^{31}-1$	Integer
long	64 bits	-2^{63}	$2^{63}-1$	Long
float	32 bits	$(+/-)1.4\ 10^{-45}$	$(+/-)3.4\ 10^{38}$	Float
double	64 bits	$(+/-)4.9\ 10^{-324}$	$(+/-)1.8\ 10^{308}$	Double
void	-	-	-	Void

Les primitives : Casting

Casting en Java

1. Casting implicite — sur-casting

Un sur-casting peut être effectué d'une manière implicite.

```
int a = 6;       // int : 32 bits
long b;         // long : 64 bits
b = a;           // Casting implicite
```

2. Casting explicite

Un casting explicite consiste à faire précéder la valeur par le nouveau type entre parenthèses :

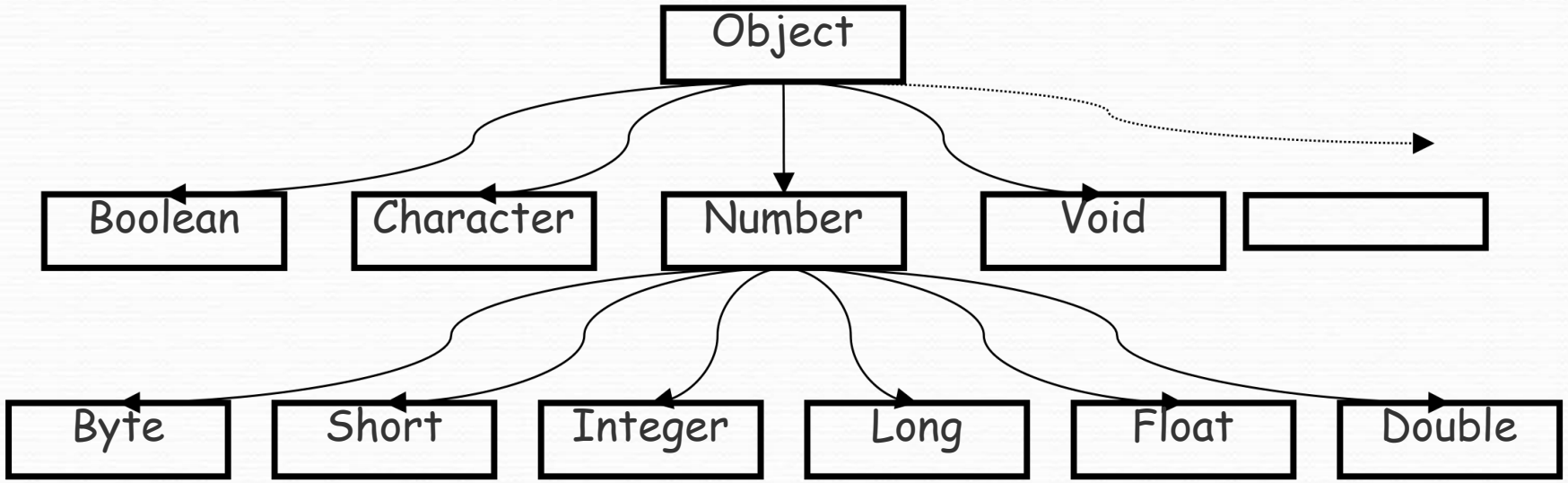
```
long b;
int a = 6;
b = (long) a;
```

Exemples :

```
float a = (float) 5.5;
double c = (double) a;
int d = 8;
byte f = (byte) d;
```

Les objets simples

- Tout objet Java hérite de la classe Object :



Primitives <-> Objets

- Passer d'une primitive à l'objet correspondant :

```
int i = 5;
Integer obj = new Integer (i);
```

- Dans la classe Number sont définies les méthodes de conversion intValue, doubleValue, floatValue, ...

```
int j = obj.intValue();
```

- Dans la classe Object est définie la méthode toString retournant la valeur d'un objet sous forme de chaîne.

Les opérateurs

- Arithmétiques :

  pour tout nombre x, on peut écrire :
  x++; x--; ++x; --x; x+=2; x-=2; x*=2; x/=2;
  x%=2 (x = x modulo 2)

  <, <=, >, >=, == égalité

  != : différence; ! : négation

- Logiques : pour tout booléen x et y,

  non x ⇔!x
  x et y ⇔x & y ou x && y *(test économe)*
  x ou y ⇔x | y ou x || y *(test économe)*
  x ou exclusif y ⇔x ^ y

Priorité des opérateurs

N°	Catégorie	Opérateurs
1	Opérateurs postfix	`expr++ expr--`
2	Opérateurs unaires	`++expr --expr +expr -expr ~ !`
3	Multiplication, division, modulo	`* / %`
4	Addition et soustraction	`+ -`
5	Décalage	`<< >> >>>`
6	Comparaison	`< > <= >= instanceof`
7	Égalité	`== !=`
8	OU exclusif	`^`
9	ET	`&`
10	OU	`|`
11	ET logique	`&&`
12	OU logique	`||`
13	Ternaire	`? :`
14	Assignement	`= += -= *= /= %= ^= |= <<= >>= >>>=`
15	Arrow	`->`

Entrées clavier

Scanner : méthodes de lecture

Méthode	Retour	Rôle
`nextLine()`	String	Lit une ligne complète
`next()`	String	Lit un mot (jusqu'à l'espace)
`nextInt()`	int	Lit un entier
`nextDouble()`	double	Lit un réel
`nextBoolean()`	boolean	Lit true ou false
`hasNextInt()`	boolean	Teste si le prochain mot est un entier

```
Scanner sc = new Scanner(System.in);
System.out.print("Nom : ");
String nom = sc.nextLine();
System.out.print("Age : ");
int age = sc.nextInt();
System.out.println(nom + " a " + age + " ans");
sc.close();
```

Piège : nextInt() puis nextLine()

```
int age = sc.nextInt();          // lit 25, laisse le
retour à la ligne
String nom = sc.nextLine();    // renvoie "" (fin de la
ligne précédente)
// Solution 1 : vider la ligne
int age = sc.nextInt();
sc.nextLine();
String nom = sc.nextLine();
```

Pourquoi ? nextInt() ne consomme pas le caractère de fin de ligne, que nextLine() lit ensuite.

Console

```
Console c = System.console();    // null si lancé depuis
un IDE
String login = c.readLine("Login : ");
char[] mdp = c.readPassword("Mot de passe : ");
```

Structures de contrôle

- Les structures de contrôle utilisées en Java sont fortement inspirées de celles du langage C ou C++ :
  - if ; else ; while ; do ; for ; break ; goto

Branchement conditionnel (1/2)

- Si … Alors … Sinon

```
if (test)
      {
       action1_si_test_vrai;
       action2_si_test_vrai;
      }
else
      {
       action1_si_test_faux;
       action2_si_test_faux;
      }
```

- Le test est de la forme :

  a ; !a ; a & b ; a & !b ;

  x<y ; x == y; x != y ; …

Branchement conditionnel (2/2)

- Branchement selon un choix multiple :

```
switch (expression)
{
    case valeur_1 : actions; break;
    case valeur_2 : actions; break;
    case valeur_3 : actions; break;
    default : actions si valeur inconnue;
}
```

Boucle "Tant que"

- **while** (test)
  {
  actions;
  }

- **do**
  {
  actions;
  }
  while (test);

Boucle "Pour"

- int i;
  for(i=0; i<5; i++) // *ou* ***for***(*int i=0; i<5; i++*)
  {
  actions;
  }

Sorties de boucles

- L'instruction **break** permet de sortir d'une boucle avant sa fin "normale".
- L'instruction **continue** passe le reste de la boucle, mais n'en sort pas.
- L'instruction **goto label** subsiste encore. Le label doit alors être défini (et suivi de **:**).

Les tableaux

Un tableau contient un ensemble fini d'éléments d'un même type. Les éléments sont accessibles par un indice ; le premier élément possède l'indice 0.

En Java, les tableaux sont des objets dérivés de Object.

Ils peuvent utiliser des méthodes héritées comme equals() et getClass().

Déclaration et allocation

Deux syntaxes de déclaration sont possibles :

```
int tableau[] = new int[50];

int[] tableau = new int[50];
```

La déclaration et l'allocation peuvent être séparées :

```
int tab[];
tab = new int[50];
```

Passage d'un tableau a une méthode

Un tableau peut être passé comme paramètre d'une méthode.

Exemple :

```
public void afficher(String texte[]) {
    // ...
}
```

Tableaux multidimensionnels

Java ne supporte pas directement les tableaux multidimensionnels.

On utilise un tableau de tableaux.

Une paire de crochets est utilisée pour chaque dimension.

```
float tableau[][] = new float[10][10];
```

Initialisation automatique

Chaque élément du tableau est initialisé selon son type par l'instruction new :

0 pour les numériques

'\0' pour les caractères

false pour les booléens

null pour les chaînes de caractères et les autres objets

Initialisation explicite

```
int tableau[5] = {10, 20, 30, 40, 50};
int tableau[3][2] = {{5, 1}, {6, 2}, {7, 3}};
int tableau[] = {10, 20, 30, 40, 50};
int[][] tabEntiers = {{1, 2, 3, 4, 5, 6},{1, 2, 3, 4},
 {1, 2, 3, 4, 5, 6, 7, 8, 9}};
```

Le parcours d'un tableau

La variable length retourne le nombre d'éléments du tableau.

Il est alors possible d'utiliser une boucle pour itérer sur chacun des éléments.

Exemple :

```
for (int i = 0; i < tableau.length; i++) {
    // ...
}
```

Parcours avec for-each

```
for (type element_courant : type[] ) {
    // code à exécuter sur l'élément courant identifié
par element_courant
}
```

Exemple :

```
String[] chaines = {"element1","element2","element3"};
for (String chaine : chaines) {
    System.out.println(chaine);
}
```

Concepts de l’orienté objet avec java

Exemple de programme

- Programme affichant « Bonjour !!!». Ce programme contient une classe principale Bonjour. Il doit donc se nommer Bonjour.java

```
public class Bonjour
{
        public static void main(String []  args)
        {
                System.out.println(“Bonjour !!! “);
        }
}
```

Exemple de programme

- Créer un fichier source Bonjour.java

  Un fichier source contient du texte, écrit en Java

  Compiler le source en fichier bytecode Bonjour.class

  Le compilateur **javac**, traduit le texte source en instructions compréhensibles par la Machine Virtuelle Java (JVM)

  Exécuter le programme contenu dans le fichier **bytecode**

  L'interprète java implémente la **JVM**

  L'interprète traduit le bytecode en instructions exécutables par la machine

Exemple de programme

Fichier Bonjour.java

```
Public class Bonjour {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

Compilation

javac Bonjour.java



Bonjour.class

Exécution

java Bonjour

Hello World !

La compilation

- La compilation d'un programme, ne génère pas d'instructions spécifiques à votre plate-forme
- Mais du **bytecode** Java, qui sont des instructions de la **Machine Virtuelle Java** (JVM)
- Si votre plate-forme (Windows, UNIX, MacOS, un browser Internet) dispose d'une JVM, elle peut comprendre le bytecode

Exemple de programme

Définir la classe `Bonjour`

```
public class Bonjour {
  public static void main(String[] args) {
      System.out.println("Bonjour !");
  }
}
```

La méthode `main`

```
public class Bonjour {
  public static void main(String[] args) {
      System.out.println("Bonjour !");
  }
}
```

Une application Java doit contenir une méthode main

- Appelée en premier par l'interprète
- main appelle les autres méthodes nécessaires pour exécuter l'application

Classes et de Méthodes

« **class Bonjour** » est la déclaration d'une classe (obligatoire en Java).
« **public** » signifie que main est visible de l'extérieure de la classe Bonjour.
« **static** » signifie que main n'est pas associée a une instance mais à une classe…
« **void** » main ne retourne rien

« **main** » est le nom de la méthode…
« **argv** » est le tableau de chaînes de caractères habituels du C.
« **String** » est le nom de la classe chaîne de caractères
« **System** » est le nom de la classe système
« **out** » est le nom de l'instance pour effectuer des sorties
« **println** » est le nom de la méthode qui imprime une ligne avec un retour chariot.

Classes et de Méthodes

Dans une fichier texte **Nomdelaclasse.java** on écrira

```
class Nomdelaclasse {
// déclaration des attributs
Type nom ;
//méthodes
Type retour Nom (type arg1, type arg2) {
}… ;

//fonction main
```

S'il n'y a pas de type retour (par exemple si on fait un affichage ou si on travaille sur les attributs) on met **void.**

Notion de Classe

Concept

- Une classe est un support d'encapsulation : c'est un ensemble de données et de fonctions regroupées dans une même entité.
- Une classe est une description abstraite d'un objet.
- Les fonctions qui opèrent sur les données sont appelées des méthodes.

Notion de Classe

- Instancier une classe consiste à créer un objet sur son modèle. Entre classe et objet il y a, en quelque sorte, le même rapport qu'entre type et variable.

- En Java : tout appartient à une classe sauf les variables de types primitifs (int, float…).

- Pour accéder à une classe il faut en déclarer une instance de cette classe (ou un objet).

Par convention:

- Les noms de classe commencent par une Majuscule.
- Les noms d'instances ou de variables commencent par une minuscule

Notion de Classe

Une classe comporte sa déclaration, des variables et la définition de ses méthodes

public : La classe est accessible partout

Private : La classe n'est accessible qu'à partir du fichier où elle est définie

protected : Est accessible aux sous-classes et aux classes voisines,

private protected : Est accessible également aux sous-classes,

Notion de Classe

final : La classe ne peut pas être modifiée, sa redéfinition grâce à l'héritage est interdite. Les classes déclarées final ne peuvent donc pas avoir de classes filles.

abstract : La classe contient une ou des méthodes abstraites, qui n'ont pas de définition explicite.

Une classe déclarée abstract ne peut pas être instanciée : il faut définir une classe qui hérite de cette classe et qui implémente les méthodes nécessaires pour ne plus être abstraite.

Autres mots-clés : final

- **final** définit :
  - un attribut invariable,
  - une méthode qui ne peut être surchargée,
  - une classe qui ne peut être héritée.

Autres mots-clés : static

- **static** définit un membre commun à toutes les instances de la classe
- On parle de membre de classe
  par opposition à membre d'instance
- Une variable de classe est indépendante de l'objet, elle existe dès le premier appel à la classe.

Autres mots-clés : static

- Seule une méthode de classe peut utiliser des variables de classe
- Une méthode de classe ne peut être surchargée dans les sous-classes
- Utilisation des membres de classe : compteur d'objet, optimisation (définition de constante)

Autres mots-clés : abstract

- abstract associé à une méthode permet de ne pas lui donner de corps. Elle devra donc être définie dans les sous-classes.
- Si une méthode est abstraite, sa classe doit être définie comme abstraite.
- Une classe abstraite ne peut être instanciée.

Autres mots-clés : abstract

- L'abstraction : définir un moule général pour les classes dérivées.
- Une **interface** est une classe :
  - ne contenant que les entêtes de méthodes
  - ne contenant pas d'attributs variables.
  - Notation :
    interface Etre_vivant {…}
    class Animal **implements** Etre_vivant {…}

Attributs et Méthodes

Les Attributs

Les données d'une classe sont contenues dans les propriétés ou attributs. Les attributs sont les données d'une classe, ils sont tous visibles à l'intérieur de la classe.

Syntaxe générale:

[< modificateur de visibilité>] <type> <nom_attribut> [=<expression>] ;

Exemple

```
class Rectangle{
private int longueur ;
private int largeur ;
public Point Centre ;
}
```

Attributs et Méthodes

Les variables d'instances

Une variable d'instance nécessite simplement une déclaration de la variable dans le corps de la classe.

Exemple :

```
public class MaClasse {
public int valeur1 ;
int valeur2 ;
protected int valeur3 ;
private int valeur4 ;
}
```

Chaque instance de la classe a accès à sa propre occurrence de la variable.

Attributs et Méthodes

Les variables de classes (static)

Les variables de classes sont définies avec le mot clé **static**

Exemple :

```
public class MaClasse {
static int compteur ;
}
```

Chaque instance de la classe partage la même variable.

Attributs et Méthodes

Les constantes

Les constantes sont définies avec le mot clé **final** : leur valeur ne peut pas être modifiée.

Exemple :

```
public class MaClasse {
final double pi=3.14 ;
}
```

Attributs et Méthodes

Les méthodes

Les méthodes sont des fonctions qui implémentent les traitements de la classe.

La syntaxe de la déclaration

La syntaxe de la déclaration d'une méthode est :

```
modificateurs type_retourné nom_méthode ( arg1, ... ) {... //
Définition des variables locales et du bloc d'instructions
// Les instructions
}
```

Le type retourné peut être élémentaire ou correspondre à un objet. Si la méthode ne retourne rien, alorson utilise **void.**

Attributs et Méthodes

Une méthode représente l'équivalent d'une procédure ou fonction dans les langages classiques de programmation.

Exemple :

```
class Rectangle {
private int longueur ;
int largeur ;
public int calcul_surface ( ) {
return (longueur*largeur) ;
}
public void initialise (int x, int y) {
longueur =x ; largeur = y ;
}
}
```

Attributs et Méthodes

Toutes les méthodes sauf les «constructeurs » doivent avoir un type de retour.

- Les méthodes qui ne renvoient rien, utilisent « **void** » comme type de retour.

- Une méthode peut ne pas avoir besoin d'arguments, ni de variables à déclarer.

- Les méthodes agissent sur les attributs définis à l'intérieur de la classe.

- Sans modificateur, la méthode peut être appelée par toutes autres méthodes des classes du package auquel appartient la classe.

- La valeur de retour de la méthode doit être transmise par l'instruction **return.**

Exemple :

```
int add(int a, int b) {
    return a + b;
}
```

La transmission de paramètres

La transmission de paramètres

- Lorsqu'un objet est passé en paramètre, ce n'est pas l'objet lui même qui est passé mais une référence sur l'objet.

- La référence est bien transmise par valeur et ne peut pas être modifiée mais l'objet peut être modifié via un message (appel d'une méthode).

- Pour transmettre des arguments par référence à une méthode, il faut les encapsuler dans un objet qui prévoit les méthodes nécessaires pour les mises à jour.

Constructeurs

Un constructeur d'objet est une méthode particulière d'une classe qui est appelée lors de la création d'un objet, quel qu'il soit. Son but est :

- d'allouer un emplacement mémoire pour l'objet,

- d'initialiser les attributs de l'objet avec de bonnes valeurs de départ

- de retourner l'adresse de l'emplacement mémoire choisi.

Destructeur

Destructeur

Il n'y a pas de destructeur en Java. Cependant, un destructeur d'objet est une méthode très commune en POO qui :

(d) remet l'objet dans un état terminal et

(e) libère l'emplacement mémoire associé à l'objet.

Ramasse Miettes

- En JAVA, tout est objet. Il faut donc créer explicitement chaque objet manipulé.
- Java n'a pas de destructeur d'objet. Java possède un « ramasse-miettes » qui libère les emplacements mémoire occupés par les objets non référencés
- Cependant, il est inutile de les détruire. Le Garbage Collector (ramasse miettes) s'en charge.
- Dès qu'un objet n'est plus utilisé (sortie de boucle, ...), il est détruit.

le mot-clé *this*

- Quand le programme est dans une méthode d'instance, le programmeur peut manipuler La référence de l'instance courante avec le mot-clef '**this**'.
- Le champ **this** d'une classe se réfère à l'instance de l'objet en cours de traitement.
- Il peut être utilisé : Dans un constructeur
- Lors d'un appel à une fonction d'une autre classe requérant l'objet
  Pour retourner une référence à l'objet

```
class Essai { ....
    Essai changeNom(String nom2)
    {
       nom = nom2;   return this;
    }
}
```

Exemples

Exemple : une classe qui permet de faire la somme du contenu de deux entiers stockes dans des variables pour la stocker dans une troisième variable

```
class Somme {
// attributs
int x ;
int y ;
int s ;
// methode qui somme le contenu de x et de y et
//  qui renvoie leur somme , le type retour est donc int
        int somme ( ) {
        return x+y;
        }
// methode qui somme le contenu de x et de  y . Le résultat n'est pas
retourne //mais il est stocke dans la troisième variable.
        void addition ( ) {
        s=x+y ;
        }
}
```

Exemple 2 : La portée des variables (qu'affiche le programme?)
Une variable existe dans le bloc dans lequel elle a été créée.

```
Class Exemple {
// attribut
int x=10 ;
// methode qui crée une variable y stocke 5
//dedans et retourne la valeur.
        int methode () {
        int y=5 ;              //y est une variable locale
        return y ;
        }
// methode qui affiche
        void affichage ( ) {          // on affiche x car x est une variable globale
        System.out.println (x) ;      //??????
         System.out.println(y);}}
```

Getters et setters

Accéder aux attributs privés tout en respectant l'encapsulation

- Les attributs sont déclarés private : l'extérieur ne peut ni les lire ni les modifier directement
- Getter (accesseur) : méthode qui retourne la valeur d'un attribut
- Setter (mutateur) : méthode qui modifie la valeur d'un attribut
- La classe garde le contrôle : validation, lecture seule, changement de représentation interne sans casser le code appelant

```
public class Compte {
    private double solde;      //
inaccessible de l'extérieur

    public double getSolde() {          //
getter
        return solde;
    }
    public void setSolde(double solde) { //
setter
        this.solde = solde;
    }
}

Compte c = new Compte();
// c.solde = 100;        // ERREUR : solde
est private
c.setSolde(100);
System.out.println(c.getSolde());   // 100.0
```

Conventions de nommage et syntaxe

```
public class Personne {
    private String nom;
    private int age;
    private boolean actif;

    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom =
nom; }

    public int getAge() { return age; }
    public void setAge(int age) {
        if (age >= 0 && age <= 150) this.age = age;
// validation
    }
    public boolean isActif() { return actif; }
// boolean : is
    public void setActif(boolean actif) {
this.actif = actif; }
}
```

- getX() : sans paramètre, retourne le type de X
- setX(valeur) : void, un paramètre
- boolean : isX() au lieu de getX()
- this.x = x pour distinguer attribut et paramètre
- Les IDE les génèrent automatiquement

Contrôler l’accès et pièges à éviter

Besoin	Solution
Lecture seule	getter sans setter
Validation des valeurs	test dans le setter (ou exception)
Attribut calculé	le getter calcule la valeur à la volée
Objet immuable	champs final, getters seulement

```
private int[] notes = {12, 15};

public int[] getNotes() {
    return notes.clone();   // copie
défensive
}
// sans clone(), l'appelant pourrait
// modifier le tableau interne !
```

En Java : un setter sur chaque attribut sans réflexion annule l’intérêt de l’encapsulation. Un getter qui renvoie un objet modifiable (tableau, liste) doit renvoyer une copie.

Constructeur, Ex: 1 Class Point

```
/** Modélise un point de coordonnées x, y */
public class Points {
private int x, y;
   public Points(int x1, int y1) { // un constructeur
   x = x1;
   y = y1;
   }
public double distance(Points p) { // une méthode
   return Math.sqrt((x-p.x)*(x-p.x) + (y-p.y)*(y-p.y)); }

public static void main(String[] args) {
   Points p1 = new Points(1, 2); // Création deux objets
   Points p2 = new Points(5, 1);
   System.out.println("Distance : " + p1.distance(p2)); }


}
```

2 classes et 1 fichier

```
/** Modélise un point de coordonnées x, y */
public class Points {
private int x, y;
    public Points(int x1, int y1) {
    x = x1; y = y1;
    }
    public double distance(Points p) {
    return Math.sqrt((x-p.x)*(x-p.x) + (y-p.y)*(y-p.y));
    }
}
/** Teste la classe Points */
class TestPoints {
    public static void main(String[] args) {
    Points p1 = new Points(1, 2);
    Points p2 = new Points(5, 1);
    System.out.println("Distance : " + p1.distance(p2)); }
}
```

Compilation et exécution

- Programme source Java = ensemble de fichiers « **.java** »

- Chaque fichier « **.java** » contient une ou *plusieurs* définitions de classes

- Au plus une définition de classe **public** par fichier « .java » avec nom du fichier = nom de la classe publique

Compilation et exécution

- Durant l'exécution d'un code Java, les classes

(leur *bytecode*) sont chargées dans la **JVM** au fur et à mesure des besoins

- Une classe peut être chargée:

– depuis la machine locale (le cas le plus fréquent)

– depuis une autre machine, par le réseau

– par tout autre moyen (base de données,…)

2 classes dans 2 fichiers

Fichier **Point.java**

```
/** Modélise un point de coordonnées x, y */
public class Points {
private int x, y;
   public Points(int x1, int y1) {
   x = x1; y = y1;
   }
   public double distance(Points p) {
   return Math.sqrt((x-p.x)*(x-p.x) + (y-p.y)*(y-p.y));
   }
}
```

Fichier **TestPoints.java**

```
/** Pour tester la classe Points */
class TestPoint {
   public static void main(String[] args) {
   Points p1 = new Points(1, 2);
   Points p2 = new Points(5, 1);
   System.out.println("Distance : " + p1.distance(p2));
   }
}
```

2 classes et 1 fichier

```
/** Modélise un point de coordonnées x, y */
public class Points {
private int x, y;
    public Points(int x1, int y1) {
    x = x1; y = y1;
    }
    public double distance(Points p) {
    return Math.sqrt((x-p.x)*(x-p.x) + (y-p.y)*(y-p.y));
    }
}
/** Teste la classe Points */
class TestPoints {
    public static void main(String[] args) {
    Points p1 = new Points(1, 2);
    Points p2 = new Points(5, 1);
    System.out.println("Distance : " + p1.distance(p2)); }
}
```

Plusieurs constructeurs (surcharge)

```
public class Employe {
private String nom, prenom;
private double salaire;
// 3 Constructeurs
   public Employe() {}
   public Employe(String n, String p) {
   nom = n;
   prenom = p;
   }
   public Employe(String n, String p, double  s) {
   nom = n;
   prenom = p;
   salaire = s;
   }
. . .
Em1 = new Employe("Dupond", "Pascal");
Em2 = new Employe("Durand", "Jacques", 12000);
```

Constructeur par défaut

Lorsque le code d'une classe ne comporte pas de constructeur, un constructeur sera automatiquement ajouté par Java

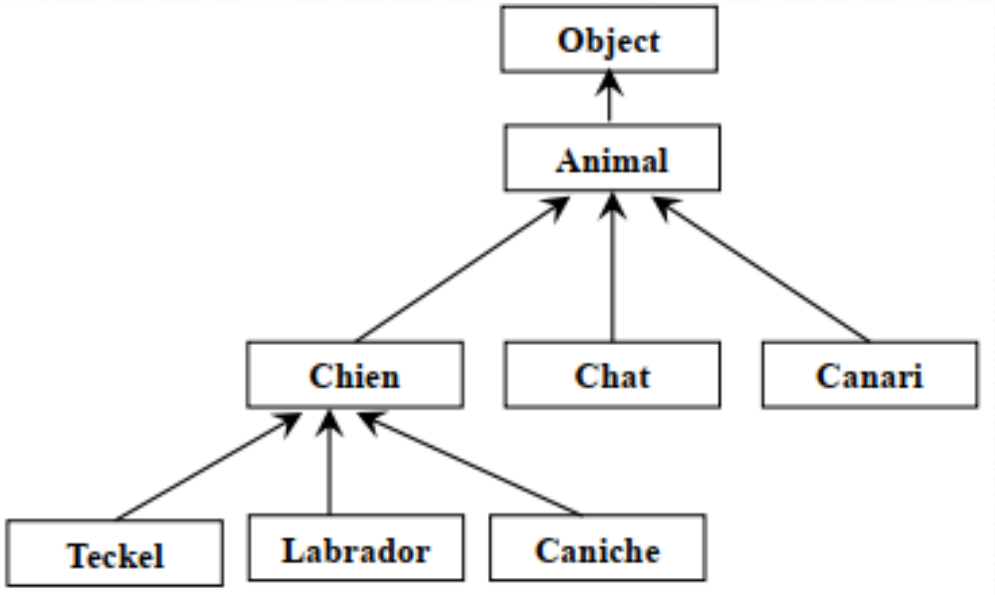
Pour une classe **Classe**, ce constructeur sera par défaut défini par Java

[**public**] **Classe**() { }



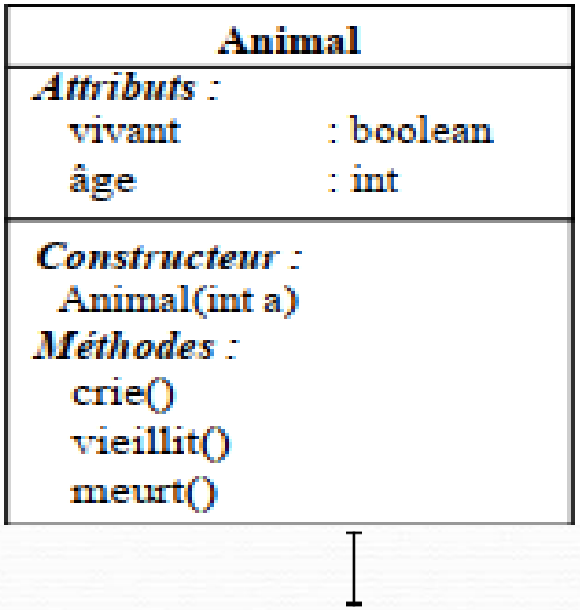
Concept de Héritage

Considérons le modèle suivant :



Concept de Héritage

La classe Animal peut s ’écrire :




```
class Animal {
boolean vivant;
int âge;
Animal (int a) {
âge = a;
vivant = true;
}
void vieillit() {
++âge;
}
void meurt() {
vivant = false;
}
void crie() {
}
}
```

Dans ce cas, la création d'un Animal se ferait à l'aide de l'instruction suivante : Animal nouvelAnimal = new Animal(3);

Concept de Héritage

- La classe Canari hérite de la classe Animal.
- Pour faire l 'héritage avec java, on emploi la commande extends.

```
class Canari extends Animal{ }
```

- La classe canari hérite de la classe Animal tous ses membres sauf le constructeur. Il faudra doc redéfinir le constructeur de la classe Canari.

Concept de Héritage

❑ **Référence à la classe parente:**

✓ Pour éviter de réécrire le même code du constructeur de la classe Animal, on fera référence à la classe parente en utilisant le mot super

✓ La classe Canari devient:

```
class Canari extends
Animal{
Canari(int a){
super(a);
} }
```

❑ **Redéfinition des méthodes:**

✓ La classe Canari hérite de la classe Animal la méthode crie() qui est vide.

✓ On peut redéfinir cette méthode du moment que nous connaissons le cri d 'un canari.

✓ La classe Canari devient:

```
class Canari extends
Animal{
Canari(int a){
super(a);}
void crie() {
System.out.println("Cui-
cui !");
}}
```

Surcharge

❑ **Surcharger une méthode:**

✓ La classe Animal possède une méthode vieillit() sans paramètre et qui incrémente l ’âge d ’une unité .

✓ Nous pouvons ajouter à la classe Animal une autre méthode qui porte le même nom mais qui possède un paramètre a qui indique le nombre d ’années à ajouter pendant l anniversaire de l ’animal.

✓ Voici le code des deux méthodes:

```
void vieillit() {
++âge;
System.out.println("C'est
l'anniversaire de cet animal.");
System.out.println("Il a maintenant
" + âge + " an(s).");}
```

```
void vieillit(int a) {
âge += a;
System.out.println("C'est
l'anniversaire de cet animal.");
System.out.println("Il a maintenant
" + âge + " an(s).");
}
```

Surcharge

- Pour éviter de réécrire les mêmes instructions dans les deux méthodes, on peut constater que la méthode vieillit() est un cas particulier de la méthode vieillit(int a). Les deux méthodes s'écriront alors:

```
void vieillit() {
vieillit(1);
}
void vieillit(int a) {
âge += a;
System.out.println("C'est l'anniversaire de cet animal.");
System.out.println("Il a maintenant " + âge + " an(s).");
}
```

- L'appel des deux méthodes peut se faire comme suit:

✓ Animal a1=new Animal(3);
✓ a1.vieillit();
✓ a1.vieillit(6);

Surcharge

❑ **Signature d ’une méthode**

✓ La signature d'une méthode désigne la liste de ses paramètres avec leur type.

✓ Les deux méthodes vieillit(int a) et vieillit() ont le même nom et deux signatures différentes.

✓ Les deux méthodes suivantes:

- methode(int a, long b)
- methode(long b, int a)

ont deux signatures différentes.

❑ Alors que les méthodes :

✓ methode(int a, long b) et methode(int c, long d)

Surcharge

Surcharger le Constructeur.

- Les constructeurs peuvent également être surchargés.
- On peut créer deux constructeurs pour la classe Animal:

```
Animal() {
this(1);
}
Animal(int a) {
âge = a;
vivant = true;
System.out.println("Un animal de " + a + " an(s) vient d'être
créé");
}
```

- On pourra créer une instance de la classe Animal en utilisant l 'un des deux constructeurs:
  - Animal A1=new Animal();
  - Animal A2= new Animal(4);

Surcharge

Surcharger le Constructeur.

- On peut également constater que le premier constructeur est un cas particulier du deuxième.
- Un constructeur peut appeler un autre en utilisant le mot this qui désigne l'objet courant.

```
Animal() {
this(1)
}
Animal(int a) {
âge = a;
vivant = true;
System.out.println("Un animal de"+a+"an(s)créé");
}
```

- Animal A1=new Animal();
- Animal A2= new Animal(4);

Les constructeurs par défaut

- Java fournit automatiquement un constructeur par défaut.
- Le constructeur par défaut est le suivant :

NomDeLaClasse() {}

Résumé

- Pour qu'une classe hérite d'une autre, on utilise le mot extends.
- Une classe peut hériter d'une autre tout sauf les constructeurs.
- Il faut toujours définir le constructeur de la nouvelle classe dérivée.
- Le constructeur de la nouvelle classe dérivée peut faire appel au constructeur de la classe parente en utilisant le mot super suivi des valeurs des arguments entre parenthèse.
- Lors de l'héritage, la nouvelle classe dérivée peut redéfinir les méthodes héritées de la classe parente.
- Dans une classe, on peur surcharger une méthode : On peut créer plusieurs méthodes qui ont le même nom, mais avec des signatures différentes.
- Dans une classe on peut également surcharger le constructeur : On peut créer plusieurs constructeurs avec des signatures différentes.
- Un constructeur peut faire appel à un autre constructeur dans la même classe en utilisant le mot this suivi des valeurs des arguments entre parenthèses.
- Si vous ne définissez pas un constructeur dans votre classe, le compilateur java définit un constructeur par défaut.

Programme de test

```
class TestAnimaux {
public static void
    main(String[] args) {
Canari titi = new Canari(3);
Canari mimi=new Canari() ;
titi.ageMaxi=5 ;
titi.vieillit();
titi.crie();
titi.vieillit(4);
mimi.vieillit(5);
mimi.crie() ;
}
}
```

```
class Animal {
boolean vivant;
int âge;
int âgeMaxi = 10;
Animal(int a) {
âge = a;
vivant = true;
System.out.println("Un animal de " + a
+ " an(s) vient d'être créé");
}
Animal() {
this(1);
}
```

```
void vieillit(int a) {
âge += a;
if (âge > âgeMaxi)
meurt();
else{
System.out.println("C'est l'anniversaire
      de cet animal.");
System.out.println("Il a maintenant " +
      âge + " an(s)");
}
}
void vieillit() {
vieillit(1);
}
void meurt() {
vivant = false;
System.out.println("Cet animal est
      mort");
}
void crie() {
}
}
```

```
class Canari extends Animal {
Canari(int a) {
super(a);
}
Canari() {
this(1);
}

void crie() {
System.out.println("Cui-cui !");
}
}
```

Exercice : Comptes Bancaires

- Un compte bancaire possède à tout moment une donnée : son solde. Ce solde peut être positif (compte créditeur) ou négatif (compte débiteur).
- Chaque compte est caractérisé par un code incrémenté automatiquement.
- A sa création, un compte bancaire a un solde initial et un code incrémenté.
- Utiliser son compte consiste à pouvoir y faire des dépôts et des retraits. Pour ces deux opérations, il faut connaître le montant de l'opération.
- L'utilisateur peut aussi consulter le solde de son compte par la méthode getSolde().
- Faire une représentation graphique de la classe Compte
- Ecrire le code java de la classe Compte
- Ecrire le code java de la classe TestCompte qui contient la fonction main() qui exécute les actions suivantes:
  - **Créer un compte avec un solde de 40000**
  - **Créer un compte avec un solde de 3000**
  - **Faire des retraits des versements sur les deux comptes.**
  - **Afficher l'état des deux comp**